Session 3 - Optimization techniques
Geavanceerde computerarchitectuur: labosessie

(B-KUL-JLI1QS)

Elias **Neels**, Wout Batselé

March 5, 2026



Year: 2026

Class: MELEC

Instructor:

Contents

1 Context **2**

2 warps **2**

3 thread divergence **2**

4 Observations/results **3**

5 Conclusion **4**

A Output afbeeldingen **5**

B Code **6**
B.1 Part 1 6

1 Context

Het doel van dit labo is het manipuleren van een afbeelding. Dit doen we op twee manieren. We focussen daarbij op de manier waarop threads worden gegroepeerd in warps en hoe de programma architectuur de efficiëntie van deze warps beïnvloeden door thread divergence.

2 warps

Cuda beheert en voert threads uit in groepen van 32 threads, genaamd warps. Wanneer een block op een Streaming Multiprocessor (SM) wordt gestart, wordt het opgedeeld in deze warps van opeenvolgende threads.

Hierbij speelt de block-grootte een grote rol. Wanneer we bijvoorbeeld `foo<<<4, 33>>>` aanroepen, worden er per block 2 warps aangemaakt (een volledige warp van 32 threads en een tweede warp met slechts 1 actieve thread), wat leidt tot een minder efficiënt gebruik van de cores per SM. Om een afbeelding van 960×1280 pixels efficiënt te verwerken met minder threads, gebruiken we een grid-stride loop. Hierdoor lopen de threads door de data, wat de kernel onafhankelijk maakt van de exacte grid-grootte[2].

3 thread divergence

Thread divergence treedt op binnen een warp wanneer threads verschillende uitvoer tijden hebben bijvoorbeeld door verschillende paden te nemen in een conditionele branch, zoals een if-else statement.

Omdat alle threads in een warp dezelfde instructie op hetzelfde moment moeten uitvoeren (SIMT), worden de verschillende paden na elkaar (geserialiseerd) uitgevoerd. Terwijl één pad wordt uitgevoerd, zijn de threads op het andere pad inactief, wat kostbare rekenkracht verspilt. In onze Gaussian blur kernel ontstaat er divergentie omdat de rode pixels (indexen $3i$) een complexe berekening ondergaan (gewogen som van naburige pixels), terwijl de groene en blauwe pixels enkel geïnverteerd worden. Omdat deze componenten interleaved in het geheugen liggen `(RGBRGB...)`, bevinden threads met verschillende opdrachten zich in dezelfde warp[2].

We kunnen dit oplossen door de data te herformatteren naar `(RRR...GGG...BBB...)`. Zo zorgen we ervoor dat theads in een relfde warp hetzelfde pad van de conditionele branch zullen nemen en er dus minder tijd verloren gaat door wachtende theads.

4 Observations/results

Part 1

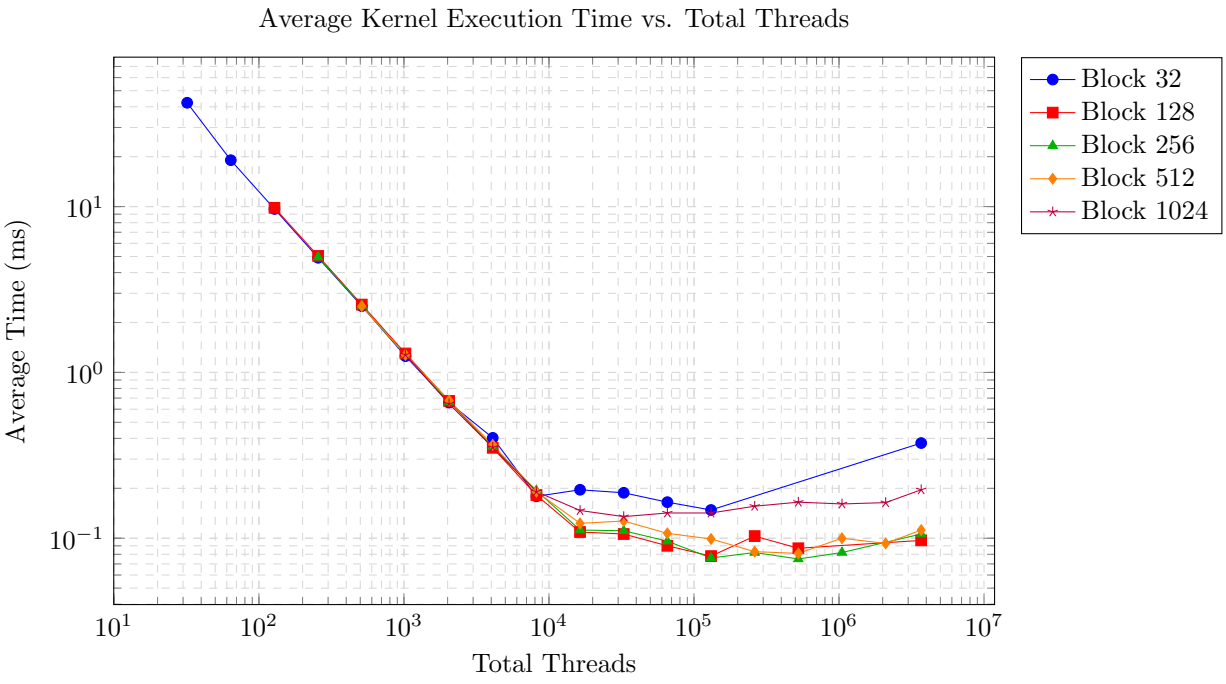


Figure 1: #threads per block size t.o.v. gemiddelde uitvoertijd

Part 2

Run	Interleaved [ms]	Planar [ms]	Speedup
1	0.202 304	0.084 064	2.41x
2	0.230 336	0.085 792	2.68x
3	0.231 104	0.178 944	1.29x
4	0.324 992	0.085 088	3.82x
5	0.213 504	0.090 176	2.37x
6	0.232 480	0.092 384	2.52x
7	0.213 216	0.085 536	2.49x
8	0.213 760	0.086 944	2.46x
9	0.275 840	0.163 424	1.69x
10	0.459 904	0.084 704	5.43x
Gemiddelde	0.259 744	0.103 706	2.50x

Table 1: Vergelijking Interleaved (Divergent) en Planar (Coherent)

a. **Warp Efficiency:** Uit de opdrachten blijkt dat een block-grootte die een veelvoud is van 32 (de warp-grootte) de beste resultaten geeft, wat de theorie bevestigd. Dit omdat er geen grotendeels lege warps worden gespawned die resources bezet houden.

b. **Impact van Divergentie:** De kernel met Gaussian blur vertoont een hogere uitvoeringstijd als de data niet geherformatteerd wordt, ook dit was al voorspeld in de theorie.

5 Conclusion

Dit labo illustreert dat men bij effectieve GPU-programmering rekening moet houden met de warp gebaseerde hardwarearchitectuur. Hoewel SIMT het schrijven van code voor onafhankelijke threads toelaat, is het minimaliseren van divergentie binnen een warp essentieel voor performantie. Door het gebruik van grid-stride loops en het optimaliseren van de data-layout (planar in plaats van interleaved) kunnen we de operaties veel sneller uitvoeren.

A Output afbeeldingen



(a) Referentieafbeelding



(b) Geïnverteerde kleuren



(c) Rode Gaussian blur en blauwe en groene inversie (Interleaved methode)



(d) Rode Gaussian blur en blauwe en groene inversie (Planaire methode)

Figure 2: Vergelijking van de referentieafbeelding met de drie verwerkte afbeeldingen voor de opdrachten.

B Code

B.1 Part 1

```

/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run `convert <name>.<extension> <name>.ppm
 *
 */
#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <device_launch_parameters.h>
#include <cuda_runtime.h>

// #define M 512        // Lenna width
// #define N 512        // Lenna height
#define M 960        // VR width
#define N 1280        // VR height
#define C 3          // Colors
#define OFFSET 16    // Header length


uint8_t* get_image_array(void){
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b
    ,M}, ..., b_{N,M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen( ./anna.ppm , rb );
    if(imageFile==NULL){
        perror( ERROR: Cannot open input file );
        exit(EXIT_FAILURE);
    }

    // Initialize empty image array
    uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(uint8_t)+OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t)+OFFSET, imageFile);

    // Close the file
    fclose(imageFile);

    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}


void save_image_array(uint8_t* image_array, const char* filename){
```

```
57      /*
58       * Save the data of an (RGB) image as a pixel map.
59       *
60       * Parameters:
61       *  - param1: The data of an (RGB) image as a 1D array
62       *
63       */
64      // Try opening the file
65      FILE *imageFile;
66      imageFile=fopen(filename, wb );
67      if(imageFile==NULL){
68          perror( ERROR: Cannot open output file );
69          exit(EXIT_FAILURE);
70      }
71
72      // Configure the file
73      fprintf(imageFile, P6\n );               // P6 filetype
74      fprintf(imageFile, %d %d\n , M, N);      // dimensions
75      fprintf(imageFile, 255\n );              // Max pixel
76
77      // Write the image
78      fwrite(image_array, 1, M*N*C, imageFile);
79
80      // Close the file
81      fclose(imageFile);
82  }
83
84
85  __global__ void invert_kernel(uint8_t* input_image, uint8_t* output_image, int n)
        {
86      int index = blockIdx.x * blockDim.x + threadIdx.x;
87      int stride = blockDim.x * gridDim.x;
88      for (int i = index; i < n; i += stride) {
89          output_image[i] = 255 - input_image[i];
90      }
91  }
92
93  __global__ void blur_interleaved_kernel(uint8_t* input_image, uint8_t*
        output_image, int n) {
94
95      int index = blockIdx.x * blockDim.x + threadIdx.x;
96      int stride = blockDim.x * gridDim.x;
97
98      for (int i = index; i < n; i += stride) {
99          if (i % 3 == 0) {
100             float r2 = (i >= 6) ? input_image[i - 6] : input_image[i];
101             float r1 = (i >= 3) ? input_image[i - 3] : input_image[i];
102             float r0 = input_image[i];
103             float r_1 = (i + 3 < n) ? input_image[i + 3] : input_image[i];
104             float r_2 = (i + 6 < n) ? input_image[i + 6] : input_image[i];
105
106             output_image[i] = (uint8_t)(0.1f * r2 + 0.25f * r1 + 0.5f * r0 + 0.25f
         * r_1 + 0.1f * r_2);
107         } else {
108             output_image[i] = 255 - input_image[i];
109         }
110     }
111 }
112
113 __global__ void blur_separated_kernel(uint8_t* input_image, uint8_t* output_image,
         int n) {
114
```

```
115     int index = blockIdx.x * blockDim.x + threadIdx.x;
116     int stride = blockDim.x * gridDim.x;
117     int num_pixels = n / 3;
118 
119     for (int i = index; i < n; i += stride) {
120         if (i < num_pixels) {
121             float r2 = (i >= 2) ? input_image[i - 2] : input_image[i];
122             float r1 = (i >= 1) ? input_image[i - 1] : input_image[i];
123             float r0 = input_image[i];
124             float r_1 = (i + 1 < num_pixels) ? input_image[i + 1] : input_image[i
    ];
125             float r_2 = (i + 2 < num_pixels) ? input_image[i + 2] : input_image[i
    ];
126 
127             output_image[i] = (uint8_t)(0.1f * r2 + 0.25f * r1 + 0.5f * r0 + 0.25f
     * r_1 + 0.1f * r_2);
128         } else {
129             output_image[i] = 255 - input_image[i];
130         }
131     }
132 }
133 
134 
135 int main (void) {
136 
137     // Read the image
138     uint8_t* image_array = get_image_array();
139 
140     // Allocate output
141     uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);
142 
143     // Device pointers
144     uint8_t *d_input, *d_output;
145     int data_size = M * N * C * sizeof(uint8_t);
146     int num_elements = M * N * C;
147 
148     // Allocate memory on device
149     cudaMalloc((void**)&d_input, data_size);
150     cudaMalloc((void**)&d_output, data_size);
151 
152     // Copy input data to device
153     cudaMemcpy(d_input, image_array, data_size, cudaMemcpyHostToDevice);
154 
155     // Part 1: Inversion with varying thread counts
156     printf( Part 1: Inversion Kernel timings\n );
157     const int NUM_RUNS = 10;
158     int test_block_sizes[] = {32, 128, 256, 512, 1024};
159     int num_block_configs = sizeof(test_block_sizes) / sizeof(test_block_sizes[0])
    ;
160     for (int b = 0; b < num_block_configs; b++) {
161         int block_size = test_block_sizes[b];
162         printf( \nTesting Block Size: %d\n , block_size);
163 
164         int max_blocks = (num_elements + block_size - 1) / block_size;
165 
166         // Warmup
167         invert_kernel<<<max_blocks, block_size>>>(d_input, d_output, num_elements)
    ;
168         cudaDeviceSynchronize();
169 
170         // Progressively increase total thread count: 1 block up to max_blocks
171         // Grid sizes beyond max_blocks are capped to avoid duplicates
```

```
172         int grid_sizes[] = {1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048,
    4096, max_blocks};
173         int num_configs = sizeof(grid_sizes) / sizeof(grid_sizes[0]);
174         int last_grid_size = -1; // track to skip duplicates after capping
175         for (int i = 0; i < num_configs; i++) {
176             int grid_size = grid_sizes[i];
177             if (grid_size > max_blocks) grid_size = max_blocks;
178             // Skip duplicate configurations caused by the cap
179             if (grid_size == last_grid_size) continue;
180             last_grid_size = grid_size;
181 
182             float total_ms = 0.0f;
183             for (int r = 0; r < NUM_RUNS; r++) {
184                 cudaEvent_t start, stop;
185                 cudaEventCreate(&start);
186                 cudaEventCreate(&stop);
187                 cudaEventRecord(start);
188                 invert_kernel<<<grid_size, block_size>>>(d_input, d_output,
    num_elements);
189                 cudaEventRecord(stop);
190                 cudaEventSynchronize(stop);
191                 float milliseconds = 0;
192                 cudaEventElapsedTime(&milliseconds, start, stop);
193                 total_ms += milliseconds;
194                 cudaEventDestroy(start);
195                 cudaEventDestroy(stop);
196             }
197             float avg_ms = total_ms / NUM_RUNS;
198             printf( Grid: %5d, Block: %4d, Total threads: %8d, Avg time: %.3f ms\n
     ,
199                     grid_size, block_size, grid_size * block_size, avg_ms);
200         }
201     }
202 
203     // Copy result back to host
204     cudaMemcpy(new_image_array, d_output, data_size, cudaMemcpyDeviceToHost);
205     // Save the image
206     save_image_array(new_image_array,  ./output_image_part1.ppm );
207 
208     // Part 2a: Interleaved data
209     printf( \nPart 2a: Interleaved (Thread Divergence)\n );
210     int optimal_block_size = 256; // Found in Part 1
211     int optimal_grid_size = (num_elements + optimal_block_size - 1) /
    optimal_block_size;
212 
213     float total_ms_2a = 0.0f;
214     for (int r = 0; r < NUM_RUNS; r++) {
215         cudaEvent_t start, stop;
216         cudaEventCreate(&start);
217         cudaEventCreate(&stop);
218         cudaEventRecord(start);
219         blur_interleaved_kernel<<<optimal_grid_size, optimal_block_size>>>(d_input
    , d_output, num_elements);
220         cudaEventRecord(stop);
221         cudaEventSynchronize(stop);
222         float milliseconds = 0;
223         cudaEventElapsedTime(&milliseconds, start, stop);
224         total_ms_2a += milliseconds;
225         cudaEventDestroy(start);
226         cudaEventDestroy(stop);
227     }
228     printf( Interleaved Avg time: %.3f ms\n , total_ms_2a / NUM_RUNS);
```

```
229
230     cudaMemcpy(new_image_array, d_output, data_size, cudaMemcpyDeviceToHost);
231     // Save part 2a image to check it's different.
232     save_image_array(new_image_array,  ./output_image_part2a.ppm );
233
234     // Part 2b: Planar data (No/Less thread divergence)
235     printf( \nPart 2b: Separated/Planar (Less Thread Divergence)\n );
236     uint8_t* separated_input = (uint8_t*)malloc(data_size);
237     uint8_t* separated_output = (uint8_t*)malloc(data_size);
238
239     // Format to separated
240     int num_pixels_img = M * N;
241     for (int i = 0; i < num_pixels_img; i++) {
242         separated_input[i] = image_array[i * 3];                              // R
243         separated_input[num_pixels_img + i] = image_array[i * 3 + 1];        // G
244         separated_input[2 * num_pixels_img + i] = image_array[i * 3 + 2];    // B
245     }
246
247     uint8_t *d_input_sep, *d_output_sep;
248     cudaMalloc((void**)&d_input_sep, data_size);
249     cudaMalloc((void**)&d_output_sep, data_size);
250     cudaMemcpy(d_input_sep, separated_input, data_size, cudaMemcpyHostToDevice);
251
252     float total_ms_2b = 0.0f;
253     for (int r = 0; r < NUM_RUNS; r++) {
254         cudaEvent_t start, stop;
255         cudaEventCreate(&start);
256         cudaEventCreate(&stop);
257         cudaEventRecord(start);
258         blur_separated_kernel<<<optimal_grid_size, optimal_block_size>>>(
    d_input_sep, d_output_sep, num_elements);
259         cudaEventRecord(stop);
260         cudaEventSynchronize(stop);
261         float milliseconds = 0;
262         cudaEventElapsedTime(&milliseconds, start, stop);
263         total_ms_2b += milliseconds;
264         cudaEventDestroy(start);
265         cudaEventDestroy(stop);
266     }
267     printf( Separated Avg time: %.3f ms\n , total_ms_2b / NUM_RUNS);
268
269     cudaMemcpy(separated_output, d_output_sep, data_size, cudaMemcpyDeviceToHost);
270
271     // Interleave back to save (optional but good to verify)
272     for (int i = 0; i < num_pixels_img; i++) {
273         new_image_array[i * 3] = separated_output[i];                        //
     R
274         new_image_array[i * 3 + 1] = separated_output[num_pixels_img + i];   //
     G
275         new_image_array[i * 3 + 2] = separated_output[2 * num_pixels_img + i];  //
     B
276     }
277
278     // Save part 2b image to check it's different.
279     save_image_array(new_image_array,  ./output_image_part2b.ppm );
280
281     // Free memory
282     cudaFree(d_input);
283     cudaFree(d_output);
284     cudaFree(d_input_sep);
285     cudaFree(d_output_sep);
286     free(image_array - OFFSET);
```

```
287     free(new_image_array);
288     free(separated_input);
289     free(separated_output);
290
291
292     return 0;
293 }
```

Listing 1: CUDA Kernel for detection of the maximum value in a array

References

[1] Feys T. Gent (2026) *CUDA_C_Programming_Guide-exercise_3*, Lab notes.

[2] Feys T. Gent (2026) *session-3*, Lab notes.